

Agilex 5 E-Series 065B 4Kp30 Camera + AI Bring-Up

Printable Linux-host procedure for the Altera Agilex 5 FPGA E-Series 065B Modular Development Kit.

Scope: A repeatable Linux-host procedure for the Altera Agilex™ 5 FPGA E-Series 065B Modular Development Kit with one or two Framos FSM:GO IMX678C MIPI CSI-2 cameras.

Design: 4Kp30 Multi-Sensor Camera with AI Inference Solution System Example Design **Official guide:** https://altera-fpga.github.io/rel-26.1/embedded-designs/agilex-5/e-series/modular/camera/camera_4k_ai/camera_4k_ai/

Release assets used: altera-fpga/agilex-ed-camera-ai, tag rel-25.1

Although the documentation is published under rel-26.1, this pre-built design's binaries and source release are rel-25.1. Use matching assets. Quartus Pro 25.1 is required to rebuild the design. A compatible newer Quartus Programmer can program the pre-built .jic; verify that it detects the carrier board before programming.

1. What this guide installs

This procedure writes:

- an HPS-first Yocto Linux image to the microSD card;
- a QSPI configuration image to the board flash;
- YOLOv8 nano detection and pose models to the board's SD-backed root filesystem.

The application starts automatically at boot. The browser UI controls the camera source and AI mode; DP carries the 4K video output with overlays.

2. Required hardware

- Agilex 5 FPGA E-Series 065B Modular Development Kit
- One or two Framos FSM:GO IMX678C cameras with suitable PixelMate MIPI CSI-2 cables
- Minimum 8 GB U3 microSD card and USB card reader
- DP display/cable capable of the required video mode (a 4K-capable display is recommended)
- Micro-USB from carrier **J35** to the host (JTAG/QSPI programming)
- Micro-USB from SOM **J2** to the host (HPS serial console)
- Ethernet from SOM **J6** to a DHCP-enabled network
- Linux x86_64 host with Internet access, Docker, and Quartus Programmer

Safety: Power off the board before inserting the SD card or connecting/reseating the MIPI camera cables. Align pin 1 on each camera cable with pin 1 on its connector.

3. Download and validate the pre-built assets

On the Linux host:

```
mkdir -p ~/agilex5-camera-4kp30
cd ~/agilex5-camera-4kp30

curl -fLO https://github.com/altera-fpga/agilex-ed-camera-ai/releases/download/rel-25.1/hps-first-vvp-isp-demo-image-agilex5_mk_a5e065bb32aes1.wic.gz
curl -fLO https://github.com/altera-fpga/agilex-ed-camera-ai/releases/download/rel-25.1/top.core.jic

echo "495605036a85bab7454ae56fabd659a4423a07e256a0ec0cbf4387270f56895c hps-first-vvp-isp-demo-image-agilex5_mk_a5e065bb32aes1.wic.gz" | sha256sum -c -
echo "8dc743444c276c5b04005d3e664011ec60cc3fa3f07f43eaa3038d0568e7c19 top.core.jic" | sha256sum -c -

gzip -dk hps-first-vvp-isp-demo-image-agilex5_mk_a5e065bb32aes1.wic.gz
```

Both checksum commands must print OK. The uncompressed .wic is the image written to the entire microSD device.

4. Write the microSD card

Identify the removable microSD device:

```
lsblk -o NAME,SIZE,MODEL,TRAN,RM,MOUNTPOINTS
```

Use the whole removable disk—for example /dev/sda—not a partition such as /dev/sda1. Verify the target by model, size, transport, and RM=1. Never select the host's NVMe/SATA disk.

Unmount the card's mounted partitions, then write the image:

```
sudo umount /dev/sdX1
sudo dd if=hps-first-vvp-isp-demo-image-agilex5_mk_a5e065bb32aes1.wic \
  of=/dev/sdX bs=4M conv=fsync status=progress
sync
sudo eject /dev/sdX
```

Replace sdX with the confirmed card device. This erases the card. After eject completes, insert the card into the SOM microSD slot.

5. Program QSPI

1. Power **off** the board.
2. Set SOM switch **S4 = OFF-OFF** for JTAG mode.
3. Connect carrier **J35** micro-USB to the host and power the board.
4. Confirm that Quartus sees the programming cable:

```
quartus_pgm -l
```

Expected: an entry similar to Agilex_5E MDK Carrier.

5. Program and verify the QSPI device:

```
cd ~/agilex5-camera-4kp30
quartus_pgm -c 1 -m jtag -o "pvi;top.core.jic"
```

Wait for Successfully performed operation(s) and 0 errors.

6. Power **off** the board, set SOM **S4 = ON-ON** (ASx4/QSPI boot), and leave the microSD installed.

6. Connect, boot, and open the HPS console

With the board powered off, connect:

- both cameras;
- DP display;
- SOM J2 micro-USB;
- SOM J6 Ethernet.

Power on the board. The J2 USB cable exposes four USB serial ports. Its **third** port is the HPS UART. If J35 is also connected, the host can show eight ports, so identify the J2 group by disconnecting and reconnecting only J2:

```
ls -l /dev/ttyUSB*
```

The four ports that disappear/reappear belong to J2. Use the third port in that group—e.g., `/dev/ttyUSB6` for group `ttyUSB4` through `ttyUSB7`.

If the host user has no serial-port permission:

```
sudo usermod -aG dialout "$USER"
```

Sign out and back in for the group change to apply. For an immediate temporary session, use `sudo`.

Open a separate terminal window with Picocom:

```
gnome-terminal --title="Agilex 5 HPS Console" -- \
  bash -lc 'sudo picocom -b 115200 --flow n /dev/ttyUSB6; exec bash'
```

Use 115200 baud, 8 data bits, no parity, one stop bit, and no flow control. Log in as `root` (no password).

7. Find the board address and open the UI

This image supplies BusyBox `ip`, which does **not** support `ip -br`. At the HPS serial console:

```
ifconfig eth0
```

Use the `inet addr` value in a host browser:

```
http://BOARD_IP/
```

For example, `http://10.42.0.12/`. The IP is DHCP-provided and may change after a reboot. The web UI should load automatically after Linux has booted.

8. Compile YOLOv8 models for the FPGA AI Suite

The pre-built image does not supply YOLO model binaries. Before downloading the models, review and accept the Ultralytics license: <https://www.ultralytics.com/license>

The release pins PyTorch 2.6.0, which does not provide a Python 3.14 wheel. A host with Python 3.14 will fail with `No matching distribution found for torch==2.6.0+cpu`. The supported, reproducible workaround below uses Ubuntu 22.04 / Python 3.10 inside Docker.

```

cd ~/agilex5-camera-4kp30
git clone --recurse-submodules -b rel-25.1 \
  https://github.com/altera-fpga/agilex-ed-camera-ai.git source

cd source/yolo_cnn
curl -fLO https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n.pt
curl -fLO https://github.com/ultralytics/assets/releases/download/v8.3.0/yolov8n-pose.pt

sudo docker run --rm -it \
  -v "$PWD:/work" \
  -w /work \
  ubuntu:22.04 \
  bash -lc '
    set -e
    export DEBIAN_FRONTEND=noninteractive
    apt-get update
    apt-get install -y --no-install-recommends \
      build-essential binutils zstd \
      cmake ninja-build \
      python3 python3-pip python3-venv python3-dev \
      libxcb1 libgl2.0-0 libgl1 libxext6 libxrender1 libsm6 libgomp1
    mkdir -p compile
    cd compile
    cmake -G Ninja ..
    ninja
  '

sudo chown -R "$USER:$USER" compile

```

The Docker dependencies include the XCB/OpenGL runtime required by OpenCV. Without them, the build can fail with `ImportError: libxcb.so.1`.

Successful output contains:

```

compile/output/generated_arch.arch/yolov8n_dla_m2m_compiled_640_384.bin
compile/output/generated_arch.arch/yolov8n-pose_dla_m2m_compiled_640_384.bin
compile/output/yolov8n_categories.txt
compile/output/yolov8n-pose_categories.txt

```

9. Install the models and reboot

Replace `BOARD_IP` with the address from `ifconfig eth0`.

```

cd ~/agilex5-camera-4kp30/source/yolo_cnn/compile/output

scp -r generated_arch.arch yolov8n_categories.txt yolov8n-pose_categories.txt \
  root@BOARD_IP:

ssh root@BOARD_IP \
  'sync && ls -l generated_arch.arch/*.bin yolov8n_categories.txt yolov8n-pose_categories.txt'

ssh root@BOARD_IP 'sync && reboot'

```

The destination is `/root` on the board and is stored on the SD-backed filesystem. Wait for the serial console login prompt, rediscover the DHCP address if necessary, and reload the web UI.

10. Final acceptance test

1. Confirm the DP monitor detects a signal and displays camera output.
2. Select camera 0 in the web UI and verify live video.
3. Select camera 1 and verify live video.
4. Select **Detect** and verify object boxes/labels.
5. Select **Pose** and verify pose overlays.
6. Reboot once more and confirm that both models and both camera sources remain usable.

With the FPGA AI Suite evaluation license, AI inference can stop after approximately 100,000 inferences. A full AI Suite license removes that limit.

Troubleshooting

Symptom	Check / corrective action
<code>quartus_pgm -l</code> shows no carrier	Recheck J35 USB cable, board power, host USB permissions, and the correct Programmer installation.
JTAG program fails	Confirm <code>S4 = OFF-0FF</code> , use the verified <code>top.core.jic</code> , and do not disconnect power during erase/program/verify.
Serial console is blank	Start the console before power-on; use the third port of the J2 four-port group; verify <code>S4 = ON-ON</code> and the microSD is seated.
Permission denied opening <code>/dev/ttyUSB*</code>	Add the host user to <code>dialout</code> , log out/in, or use <code>sudo</code> temporarily.
BusyBox rejects <code>ip -br</code>	Use <code>ifconfig eth0</code> .
<code>torch==2.6.0+cpu</code> has no matching version	The host Python is too new; use the Ubuntu 22.04 Docker command in section 8.
<code>ImportError: libxcb.so.1</code> in model build	Use the full Docker package list in section 8; it installs required OpenCV runtime libraries.
Display says "No signal"	Verify display input and DP cable, connect before board boot, reseal both ends, and power-cycle the monitor. Do not reflash QSPI solely for this symptom.
UI works but no DP output	At the HPS prompt run <code>pidof VvpIspDemo</code> and <code>systemctl --no-pager --full status vvp-isp.service</code> . Confirm the DP display link separately.

Useful references

- Altera camera example design: https://altera-fpga.github.io/rel-26.1/embedded-designs/agilex-5/e-series/modular/camera/camera_4k_ai/camera_4k_ai/
- Release assets: <https://github.com/altera-fpga/agilex-ed-camera-ai/releases/tag/rel-25.1>
- Source repository: <https://github.com/altera-fpga/agilex-ed-camera-ai/tree/rel-25.1>
- Ultralytics license: <https://www.ultralytics.com/license>